The Basic Loop

Gradient Descent, SGD & Minibatches

Lesson 2 of the Optimizer Course

The simplest optimizer there is — and the two things that make it work or blow up

In Lesson 1 we learned to "feel" a hill. Now we turn that into an actual algorithm: feel the slope, take a step, repeat. We'll run it by hand, watch it glide down a gentle bowl, then watch it zig-zag (and even explode) on a steeper one — and finally make it fast enough for real data with a trick called SGD.

Marevlo Research

1 The whole algorithm fits in one line

Remember the foggy hill. You feel the slope (∇f), and you step the opposite way (downhill). Write that down and you have **gradient descent** — the optimizer every other optimizer is a variation of:

$$w_{\text{new}} = w_{\text{old}} - \eta \nabla f(w_{\text{old}})$$

Say it in English: "your new position is your old position, minus a small step taken in the direction of the slope." The minus sign is what makes it *down*hill.

That little η ("eta") is the **learning rate** — how big a step you take. It's the single most important knob in all of deep learning, and most of this lesson is about getting it right. Everything else is just doing this one line over and over.

The big idea

Gradient descent is a three-word recipe: **feel, step, repeat.** Feel the slope, take a step downhill sized by the learning rate, repeat thousands of times. That's the entire algorithm. The art is entirely in *how big* the step should be.

2 Let's actually run it on our friendly bowl

Take the gentle bowl from Lesson 1 ($\nabla f = Aw - b$, with the true bottom at $w^* \approx (0.091, 0.636)$). Start at $w_0 = (2, 1)$, pick a modest learning rate $\eta = 0.1$, and turn the crank.

Let's actually do the numbers

Each step: compute the slope, then nudge against it by $0.1\times$ that slope.

- **Step 0:** at $(2, 1)$, slope $= (8, 3)$. New $= (2, 1) - 0.1(8, 3) = (1.20,\ 0.70)$.
- **Step 1:** at $(1.20, 0.70)$, slope $= (4.50, 1.30)$. New $= (0.75,\ 0.57)$.
- **Step 2:** slope $= (2.57, 0.46)$. New $= (0.493,\ 0.524)$.
- **Step 3:** slope $= (1.50, 0.065)$. New $= (0.343,\ 0.518)$.
- …and by step 6 we're at $(0.200,\ 0.544)$, closing in on the true bottom $(0.091,\ 0.636)$.

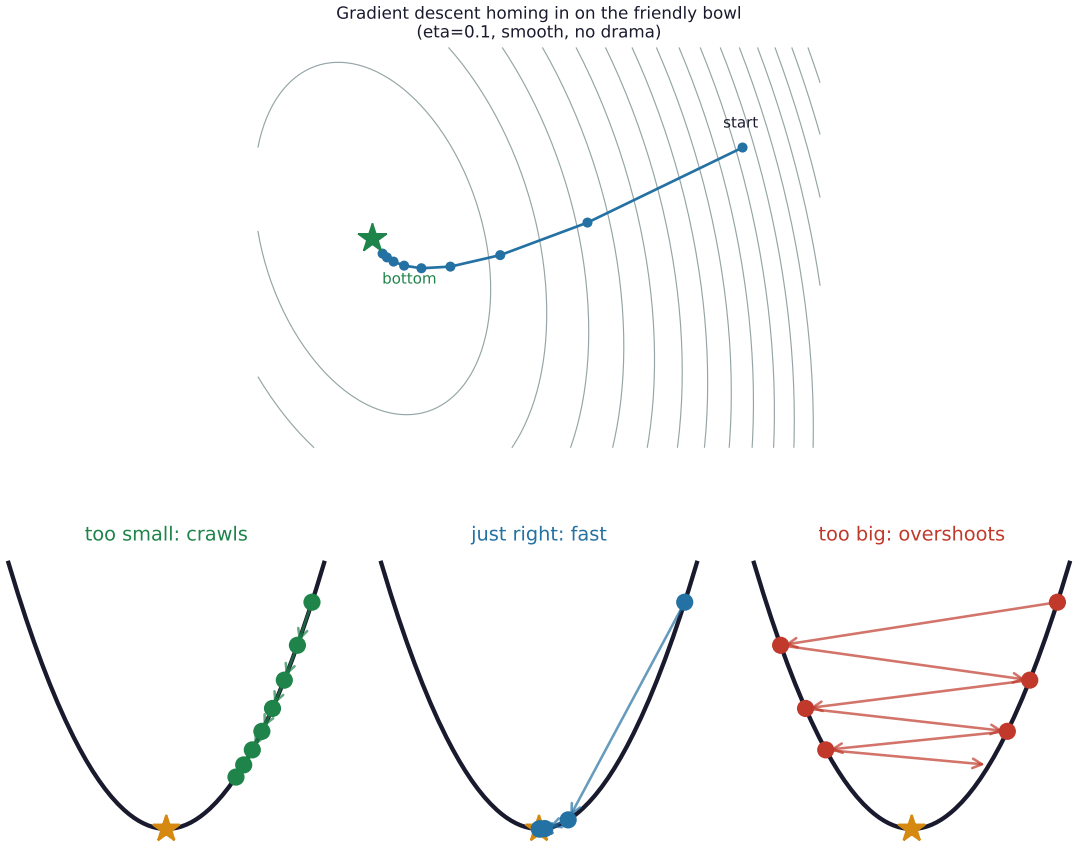
Notice the slope shrinks every step ($8 \to 4.5 \to 2.6 \to 1.5 \to \ldots$). As we near the bottom the ground flattens, so the steps naturally get smaller and we coast to a stop. No drama.

That's the happy path. It worked because the bowl was nearly round and η was sensible. Change either and things get interesting.

3 Goldilocks and the learning rate

The learning rate has to be *just right*. Here's the whole story in one picture:

- **Too small:** you inch down forever. Safe, but you'll die of old age before training finishes (and waste a fortune in compute).



- **Just right:** you stride down efficiently and settle at the bottom.
- **Too big:** you leap clean over the bottom to a *higher* point on the other side, then leap back even higher. The loss *grows*. Training "diverges" — you'll see the loss shoot to infinity or `NaN`.

How big is too big? There's an exact answer, and it's worth seeing because it connects straight back to Lesson 1. Take the simplest 1-D bowl $f(x) = \frac{1}{2}ax^2$. Its slope is ax, so one step is

$$x_{\text{new}} = x - \eta\,(ax) = (1 - \eta a)\,x.$$

Each step multiplies your position by the factor $(1 - \eta a)$. To shrink toward zero you need that factor to be smaller than 1 in size: $|1 - \eta a| < 1$, which works out to

$$0 < \eta < \frac{2}{a}.$$

Say it in English: the steeper the bowl (bigger a), the smaller your steps must be.

In many dimensions, a becomes the *steepness in each direction* — the eigenvalues from Lesson 1. Every direction must be stable, so the *steepest* one sets the speed limit:

The big idea

$$\eta < \frac{2}{\lambda_{\max}} \qquad \text{(the maximum safe learning rate).}$$

Your step size is held hostage by the single steepest direction in the whole landscape. This one inequality is why huge, lopsided models are so fiddly to train.

4 The taco strikes back: zig-zag and explosion

Now we pay off Lesson 1's warning about lopsided bowls. Take a deliberately stretched one,

$$f(w) = \tfrac{1}{2}\big(10\, w_1^2 + w_2^2\big),$$

which is $10\times$ steeper along w_1 than w_2 — condition number $\kappa = 10$. The safe limit is $\eta < 2/\lambda_{\max} = 2/10 = 0.2$. Watch what happens near that limit.

Let's actually do the numbers

$\eta = 0.18$ **(just under the limit — safe, but ugly).** Start at $(1, 1)$. The w_1 coordinate gets multiplied by $(1 - 10 \cdot 0.18) = -0.8$ each step — *negative*, so it flips sign every time. The w_2 coordinate gets multiplied by $(1 - 0.18) = 0.82$ — barely shrinking.

$$(1, 1) \to (-0.80,\ 0.82) \to (0.64,\ 0.67) \to (-0.51,\ 0.55) \to (0.41,\ 0.45) \to \cdots$$

See it? w_1 bounces $+, -, +, -$ (zig-zag across the steep walls) while w_2 crawls down agonisingly slowly. We're held to tiny progress along the long axis because the steep axis won't let us take a bigger step.

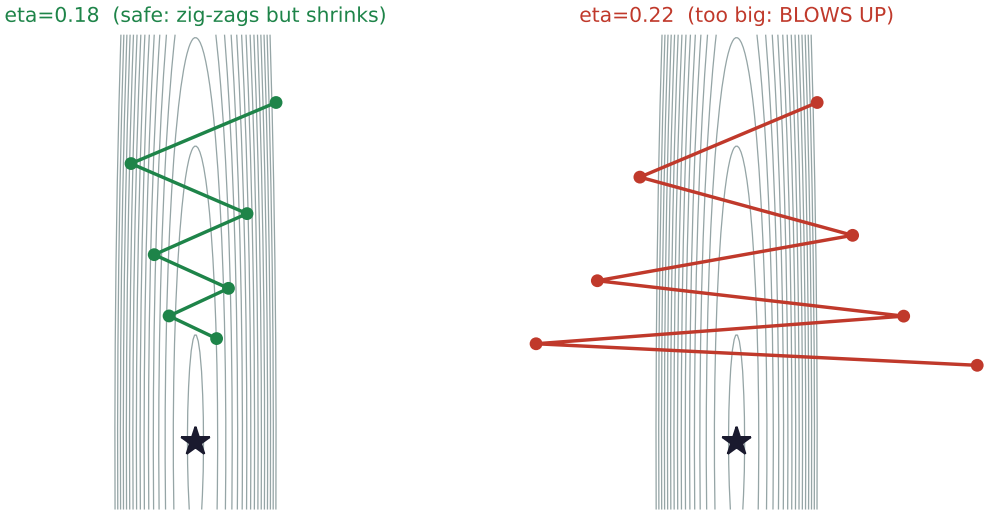
$\eta = 0.22$ **(just over the limit — disaster).** Now w_1's multiplier is $(1 - 2.2) = -1.2$, bigger than 1 in size, so it *grows* every step:

$$(1, 1) \to (-1.20,\ 0.78) \to (1.44,\ 0.61) \to (-1.73,\ 0.47) \to (2.07,\ 0.37) \to \cdots$$

w_1 explodes: $1 \to 1.2 \to 1.44 \to 1.73 \to 2.07 \to \ldots$. This is what a diverging training run looks like — a loss that climbs to infinity. A learning rate just 0.04 too large turned success into catastrophe.

When you're actually training

This is the #1 thing you'll fight in practice. If your loss zig-zags or explodes, your learning rate is too high for your landscape's steepest direction. The standard fixes: lower η (slow but safe), or use the tricks in the coming lessons — **momentum** (Lesson 3) to damp the zig-zag, or **adaptive methods** (Lessons 4–6) to give each direction its own step size so the steep one stops holding the others hostage.



5 The hidden cost: where does the gradient even come from?

We've been writing $\nabla f(w)$ as if it's free. It isn't. For real machine learning, the loss is an *average over your whole dataset*:

$$f(w) = \frac{1}{N}\sum_{i=1}^{N} \ell_i(w), \qquad \nabla f(w) = \frac{1}{N}\sum_{i=1}^{N} \nabla \ell_i(w).$$

> **Say it in English:** to compute one *exact* gradient, you must look at all N training examples and average their individual gradients.

That's fine for $N = 1000$. But modern datasets have *millions to billions* of examples. Doing all of them just to take *one* step — and you need thousands of steps — is hopeless. This all-at-once version is **Batch Gradient Descent**, and it's why it's "rarely used at scale."

5.1 The fix: estimate the gradient from a sample (SGD)

Here's the clever leap. You don't need the *exact* average slope. A rough estimate, computed from just a few examples, points roughly the same way — and roughly right, computed 1000× more often, wins easily. **Stochastic Gradient Descent (SGD)** uses the gradient from a *single* random example. **Minibatch** SGD uses a small handful (32, 64, 256…) — the universal default.

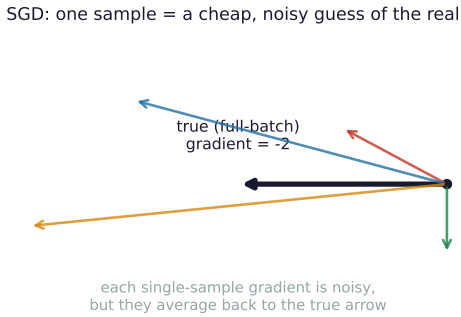
Let's actually do the numbers

Tiny example. Model $\hat{y} = w \cdot x$, four data points (x, y): $(1, 2), (2, 2), (3, 4), (4, 5)$. At $w = 1$, the predictions are $1, 2, 3, 4$, so the errors (pred − true) are -1, 0, -1, -1. The gradient from each single example is (error $\times\, x$):

$$\underbrace{-1}_{\text{ex 1}}, \quad \underbrace{0}_{\text{ex 2}}, \quad \underbrace{-3}_{\text{ex 3}}, \quad \underbrace{-4}_{\text{ex 4}}.$$

Full-batch gradient = the average = $\frac{-1+0-3-4}{4} = -2$. That's the "true" slope.

SGD picks *one* of those: maybe -1, maybe -4 — noisy guesses of the real -2. **A minibatch of two** averages two of them, e.g. examples 3&4 give -3.5, examples 1&2 give -0.5 — still noisy, but less so. More samples = less noise, but more cost. The minibatch is the sweet spot.



5.2 The surprise: the noise is a feature, not just a bug

You'd think noisy gradients are purely bad. They're not:

- **They escape saddles.** Remember the saddle villains from Lesson 1, where the slope is zero and exact gradient descent freezes? Noisy steps jiggle you off the saddle and let you keep descending.
- **They generalize better.** The noise nudges training toward *wide, flat* valleys rather than narrow, brittle ones — and flat valleys tend to mean models that work better on new data (the whole story of Lesson 8). This is partly why plain SGD still beats fancier optimizers on final test accuracy for vision.
- **They're cheap.** 1000 rough steps beat 1 perfect step, every time.

When you're actually training

You will almost never use full-batch GD. The real default is minibatch SGD (often with momentum, next lesson). Batch size is a real knob: bigger batch = less noise, smoother but slower per example, and needs a bigger learning rate; smaller batch = noisier, sometimes generalizes better. And the learning rate remains the hyperparameter you tune first, always.

6 What you can do now, and what's next

You can now state and run the core training loop, explain what the learning rate does, predict when it'll zig-zag or explode (and why the steepest direction sets the limit), and explain why we estimate the gradient from minibatches instead of computing it exactly.

Next — Lesson 3: Adding Memory. Plain GD has no memory — every step it re-decides from scratch, which is exactly why it zig-zags. What if the optimizer behaved like a heavy ball rolling downhill, building up speed in the consistent direction and smoothing out the back-and- forth? That's **momentum**, and we'll watch it crush the zig-zag on the very same stretched bowl — by hand.